



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Alejandro Mogollón Rodriguez **Carné:** 1132125

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
Es un conjunto de reglas que permite que diferentes aplicaciones de software se comuniquen entre sí a través de internet usando el protocolo HTTP.
2. ¿Qué es un endpoint?
Es una dirección digital (URL) en programación o un dispositivo físico en redes que permite la comunicación entre sistemas.
3. ¿Qué diferencia existe entre una URL y un endpoint?
La URL es solo la ruta o dirección física o lógica. El endpoint incluye la URL más la regla o método de acción que el servidor espera recibir.
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
Indican la acción específica que un cliente (como el navegador) le pide realizar a un servidor sobre un recurso en la web.
5. ¿Qué es JSON y por qué es común en las APIs?
Es un formato de texto ligero diseñado para almacenar e intercambiar datos estructurados de forma clara y fácil de leer tanto para personas como para máquinas
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
Es un número de tres dígitos que el servidor envía como respuesta a una solicitud realizada por el navegador o cliente web, indicando el resultado de dicha petición.
 - 200 OK: Indica que la solicitud del cliente se ha completado.
 - 404 Not Found: Significa que el servidor no ha podido encontrar el recurso.
 - 500 Internal Server Error: Representa un error genérico del servidor.
7. ¿Qué información se puede encontrar en la respuesta de una API?
La línea de estado HTTP, encabezados con metadatos o información técnica sobre la respuesta y servidor y el body con la información principal que se estaba solicitando.
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
 - Detectar inconsistencias en la arquitectura de datos o en el flujo de peticiones
 - Permite escribir scripts de prueba antes de programar el endpoint.
9. ¿Qué es y para qué sirve Swagger UI?
Es una herramienta de código abierto que genera una interfaz web interactiva para visualizar y probar la documentación de una API basada en la especificación OpenAPI.
10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?
Permite diseñar, enviar peticiones, probar, documentar y compartir APIs de forma visual y sin necesidad de escribir código complejo de cliente.



1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
/pet/{petId}
- Indique el método HTTP utilizado.
GET
- Indique qué parámetro necesita la solicitud.
El parámetro necesario es petId
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
Código recibido: 200 OK

- Explique con sus palabras qué información contiene la respuesta.

La respuesta muestra un JSON (también puede mostrar un XML si lo seleccionamos) con los detalles de la mascota encontrada, como el ID, nombre, entre otros.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

Parameters

Cancel

Name	Description
petId * required	ID of pet to return
integer(\$int64)	
(path)	

1

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/1' \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/1

Server response



Request URL
`https://petstore3.swagger.io/api/v3/pet/1`

Server response

Code	Details
200	Response body <pre>{ "id": 1, "category": { "id": 2, "name": "Cats" }, "name": "Cat 1", "photoUrls": ["url1", "url2"], "tags": [{ "id": 1, "name": "tag1" }, { "id": 2, "name": "tag2" }], "status": "available" }</pre> Download

Responses

Code	Description	Links
200	successful operation	No links

Media type

Controls Accept header.

200 successful operation No links

Media type

Controls Accept header.

Example Value | Schema

```
<?xml version="1.0" encoding="UTF-8"?>
<pet>
  <id>10</id>
  <name>doggie</name>
  <category>
    <id>1</id>
    <name>Dogs</name>
  </category>
  <photoUrls>
    <photoUrl>string</photoUrl>
  </photoUrls>
  <tags>
    <tag>
      <id>0</id>
      <name>string</name>
    </tag>
  </tags>
  <status>available</status>
</pet>
```

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.



POST

POST /pet Add a new pet to the store.

Add a new pet to the store.

Parameters

CancelReset

No parameters

Request body required

application/json

Create a new pet in the store

Edit Value Schema

```
{  "id": 11321,  "name": "yerbi",  "category": {    "id": 1,    "name": "Dogs"  },  "photoUrls": [    "string"  ],  "tags": [    {      "id": 0,      "name": "string"    }  ],  "status": "available"}
```

Responses

Curl

```
curl -X 'POST' \  'https://petstore3.swagger.io/api/v3/pet' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "id": 11321,  "name": "yerbi",  "category": {    "id": 1,    "name": "Dogs"  },  "photoUrls": [    "string"  ],  "tags": [    {      "id": 0,      "name": "string"    }  ],  "status": "available"  }'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet
```

Server response



Code
Details

200

Response body

```
{
  "id": 11321,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "yerbi",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

 **Download**

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 139
content-type: application/json
date: Fri, 02 Oct 2026 01:42:42 GMT
server: Jetty(9.4.53.v20231009)
```

Responses

Code	Description	Links
200	Successful operation	No links

GET posterior

Parameters Cancel

Name	Description
petId required	ID of pet to return
integer(\$int64) (path)	11321

Execute **Clear**

Responses

Curl

```
curl -X 'GET' \
'https://petstore3.swagger.io/api/v3/pet/11321' \
-H 'accept: application/json'
```



Request URL

```
https://petstore3.swagger.io/api/v3/pet/11321
```

Server response



Code

Details

200

Response body

```
{
  "id": 11321,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "Yerbi",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 139
content-type: application/json
date: Fri,02 Oct 2026 02:02:51 GMT
server: Jetty(9.4.53.v20231009)
```

Responses

Code	Description	Links
200	successful operation	No links

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Update an existent pet in the store

Edit Value | Schema

```
{
  "id": 11321,
  "name": "gerber",
  "category": {
    "id": 2,
    "name": "Cats"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Execute

Clear



Responses

Curl

```
curl -X 'PUT' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 11321,
    "name": "gerber",
    "category": {
      "id": 2,
      "name": "Cats"
    },
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 0,
        "name": "string"
      }
    ],
    "status": "available"
  }'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code Details

200

Response body

```
{
  "id": 11321,
  "category": {
    "id": 2,
    "name": "Cats"
  },
  "name": "gerber",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```



Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 140
content-type: application/json
date: Fri,02 Oct 2026 02:05:27 GMT
server: Jetty(9.4.53.v20231009)
```

Responses

Code Description

200 Successful operation

Links

No links

- ¿Qué método HTTP utilizó y por qué?
Utilicé el método PUT ya que es el adecuado para modificar o editar un registro existente, sin necesidad de eliminarlo y crearlo de nuevo.
- ¿Qué datos fueron enviados en el body?
Id: 11321
Category id: 2 (cambio respecto al original)
Category name: cat (cambio respecto al original)
Name: gerber (cambio respecto al original)
Tag id: 0
Tag name: string
Status: available
- ¿Qué código HTTP devolvió la API?
200 OK
- ¿Cómo comprobó que el cambio realmente ocurrió?
Realicé una búsqueda por medio del método HTTP: GET para corroborar que sí se reflejan cambios.



Parameters

Cancel

Name	Description
petid ★ required integer(\$int64) (path)	ID of pet to return

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/11321' \
  -H 'accept: application/json'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/11321
```

Server response

Code	Details
200	Response body <pre>{ "id": 11321, "category": { "id": 2, "name": "Cats" }, "name": "gerber", "photoUrls": ["string"], "tags": [{ "id": 0, "name": "string" }], "status": "available" }</pre> Download

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
GET /pet/findByStatus, el parámetro utilizado es el estado (status).
- Explique dónde se envía ese parámetro.
El parámetro se envía en la URL como: **/pet/findByStatus?status=available**

Request URL

```
https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available
```

- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.
Cambian los registros que se muestran, se mostrarán únicamente los que tienen su status definido como "pending".

pending

Parameters

Cancel

Name	Description
status ★ required string (query)	Status values that need to be considered for filter

Execute

Clear



Code

Details

200

Response body

```
[
  {
    "id": 1790872707950636,
    "category": {
      "id": 1,
      "name": "Dogs"
    },
    "name": "pet-1790872707950636",
    "photoUrls": [
      "https://example.com/photo.jpg"
    ],
    "tags": [
      {
        "id": 1,
        "name": "e2e"
      }
    ],
    "status": "pending"
  },
  {
    "id": 1790873482035320,
    "category": {
      "id": 1,
      "name": "Dogs"
    },
    "name": "pet-1790873482035320",
    "photoUrls": [
      "https://example.com/photo.jpg"
    ],
    "tags": [
      {
        "id": 1,
        "name": "e2e"
      }
    ],
    "status": "pending"
  }
]
```

Download

available

Parameters

Cancel

Name	Description
status * required string (query)	Status values that need to be considered for filter available

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available' \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available

Server response

Code

Details

200

Response body

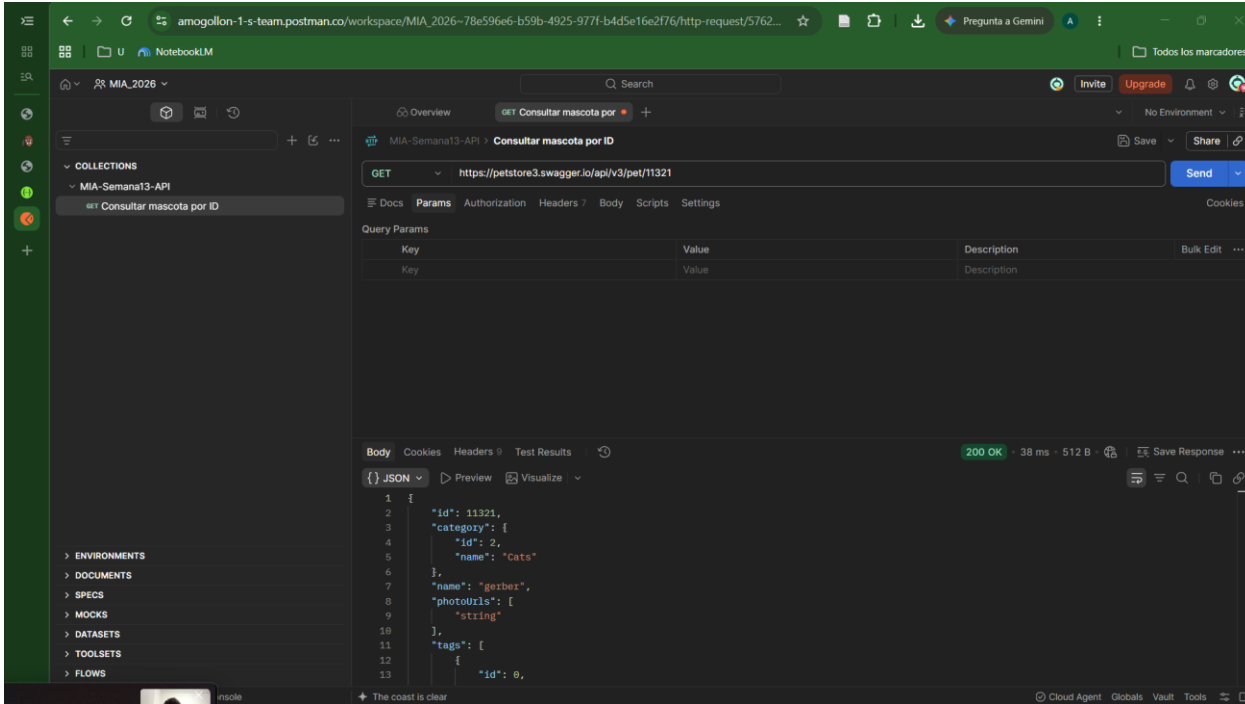
```
[
  {
    "id": 463940035,
    "name": "buzi-dog",
    "photoUrls": [],
    "tags": [],
    "status": "available"
  },
  {
    "id": 891615010,
    "name": "buzi-dog",
    "photoUrls": [],
    "tags": [],
    "status": "available"
  },
  {
    "id": 528813594,
    "category": {
      "id": 1,
      "name": "Dogs"
    },
    "name": "bella",
    "photoUrls": [
      "https://example.org/bella.png"
    ],
    "tags": [
      {
        "id": 1,
        "name": "e2e"
      }
    ],
    "status": "available"
  }
]
```

Download

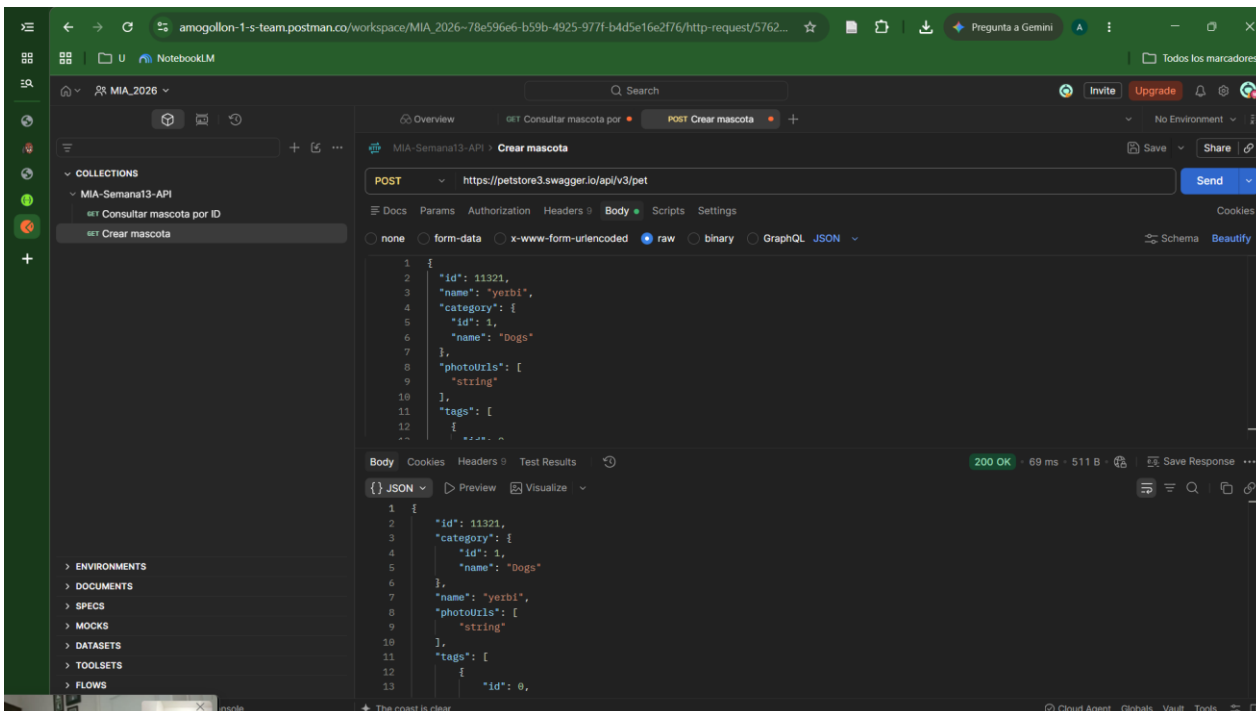


6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID



- POST – Crear mascota





- PUT – Modificar mascota

The screenshot shows a Postman interface with a PUT request to `https://petstore3.swagger.io/api/v3/pet`. The request body is a JSON object representing a pet with ID 11321, name 'gerber', and category 'Cats'. The response is a 200 OK status with a JSON object containing the updated pet information, including a photo URL and tags.

```
PUT https://petstore3.swagger.io/api/v3/pet
```

```
{
  "id": 11321,
  "name": "gerber",
  "category": {
    "id": 2,
    "name": "Cats"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,

```

```
{
  "id": 11321,
  "category": {
    "id": 2,
    "name": "Cats"
  },
  "name": "gerber",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,

```

- GET – Buscar por status

The screenshot shows a Postman interface with a GET request to `https://petstore3.swagger.io/api/v3/pet/findByStatus?status=pending`. The query parameter `status` is set to `pending`. The response is a 200 OK status with a JSON array containing one pet object with ID 1798872787958636, name 'pet-1798872787958636', and status 'pending'.

```
GET https://petstore3.swagger.io/api/v3/pet/findByStatus?status=pending
```

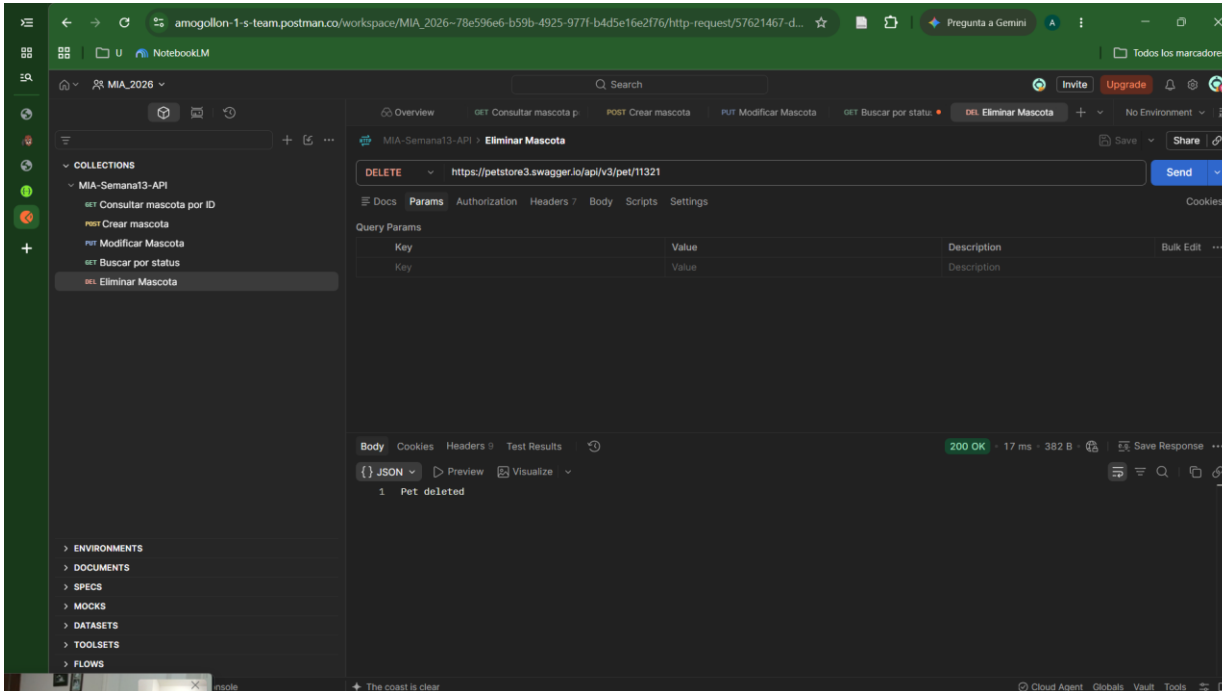
Key	Value	Description
status	pending	

```
{
  [
    {
      "id": 1798872787958636,
      "category": {
        "id": 1,
        "name": "Dogs"
      },
      "name": "pet-1798872787958636",
      "photoUrls": [
        "https://example.com/photo.jpg"
      ],
      "tags": [

```



- DELETE – Eliminar mascota



- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa



Escenario	Solicitud realizada	Código HTTP	Respuesta de la API	Lo que se esperaba	Lo que ocurrió	Explicación de la causa
1. Consultar ID inexistente	GET /pet/999999999999	Error: response status is 404	can't parse JSON. Raw result: Pet not found	Recibir un código de error de recurso no encontrado.	La API retornó un estado 404 con mensaje explícito.	El identificado r provisto no está registrado en la base de datos del servidor.
2. Enviar JSON con tipos de datos inválidos	POST /pet con "id": "nati"	Error: response status is 400	"code": 400, "message": "Input error: unable to convert input to io.swagger.petstore.model.Pet"	Recibir un error indicando invalidez en el cuerpo del JSON.	El servidor rechazó la solicitud por fallo de conversión.	El campo id espera un número entero de 64 bits (integer), no un texto/string.
3. Utilizar un parámetro con un valor no esperado	GET /pet/findByStatus?status=invalido	Error: response status is 400	"code": 400, "message": "Input error: query parameter `status value `invalido` is not in the allowable values `[available, pending, sold]`"	Que retornara un mensaje de error por status no permitido.	Retornó un mensaje de error por status no permitido.	El método valida que se ingrese uno de los 3 status, en este caso, que está registrados.
4. Eliminar un recurso ya eliminado	DELETE /pet/11321 (por segunda vez)	200 OK	can't parse JSON. Raw result: Pet deleted	Recibir un error 404, debido a que el ID ya no está registrado.	Se mostró un mensaje indicando que había sido eliminado correctamente.	Probablemente la API solo ejecuta y verifica que no exista el ID para dar una respuesta porque al intentar con un ID que no existe da el mismo resultado.



8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.

1. Registrar una mascota:

- Endpoint: /pet | Método: POST

2. Consultar su información:

- Endpoint: /pet/{petId} | Método: GET

3. Actualizar su estado:

- Endpoint: /pet | Método: PUT

4. Eliminar la mascota:

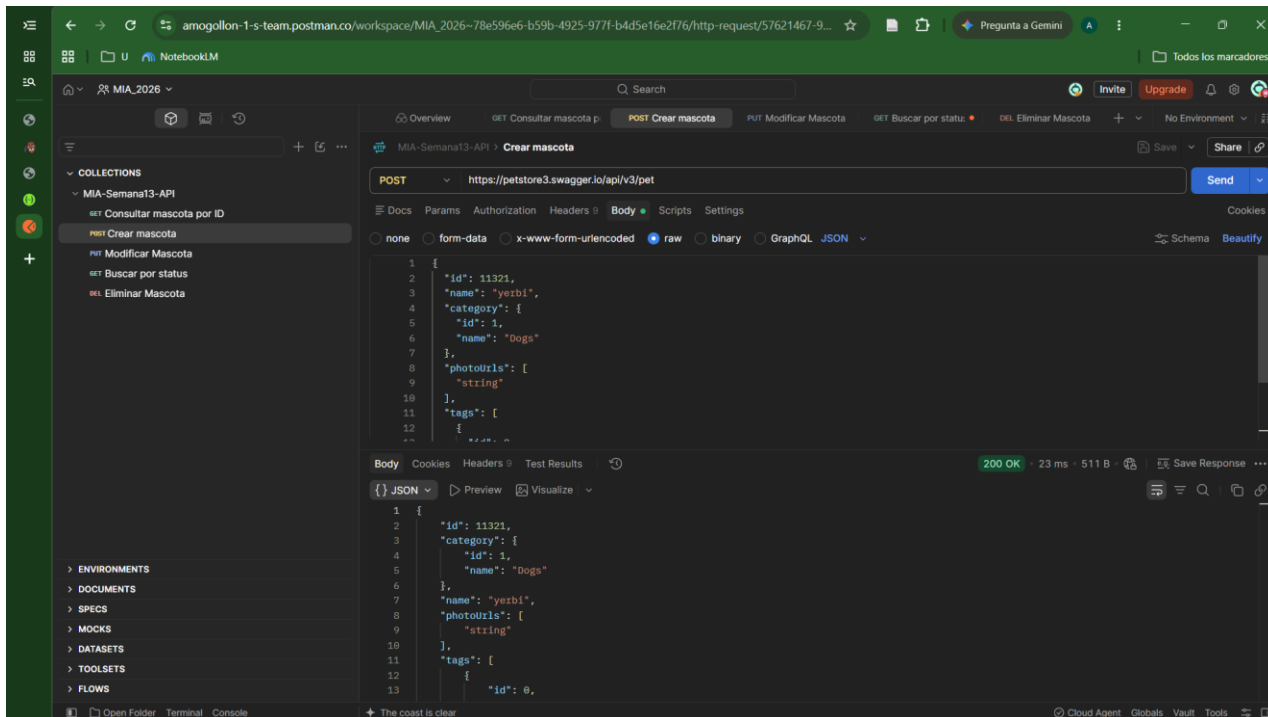
- Endpoint: /pet/{petId} | Método: DELETE

5. Identificar recurso inexistente:

- Si el sistema realiza un GET la mascota antes de ser creada o posterior a su eliminación, la API retorna el código 404 Not Found, lo que permite al software identificar que el recurso no existe.

- Ejecute el flujo completo en Postman.

Creación de la mascota





The screenshot shows the Postman interface with a GET request to `https://petstore3.swagger.io/api/v3/pet/11321`. The response is a JSON object with the following structure:

```
1 {
2   "id": 11321,
3   "category": {
4     "id": 1,
5     "name": "Dogs"
6   },
7   "name": "yerbi",
8   "photoUrls": [
9     "string"
10  ],
11  "tags": [
12    {
13      "id": 0,
```

The status bar indicates a 200 OK response with a 41 ms duration and 511 B of data.

Actualizar su estado

The screenshot shows the Postman interface with a PUT request to `https://petstore3.swagger.io/api/v3/pet`. The request body is a JSON object with the following structure:

```
7   },
8   "photoUrls": [
9     "string"
10  ],
11  "tags": [
12    {
13      "id": 0,
14      "name": "string"
15    }
16  ],
17  "status": "sold"
18 }
```

The status bar indicates a 200 OK response with a 37 ms duration and 507 B of data.



Eliminar mascota

The screenshot shows the Postman interface with a workspace named 'MIA_2026'. The left sidebar displays a collection named 'MIA-Semana13-API' with several endpoints. The main panel shows a 'DELETE' request to 'https://petstore3.swagger.io/api/v3/pet/11321'. The response is a '200 OK' status with a body of 'Pet deleted'.

Overview GET Consultar mascota p POST Crear mascota PUT Modificar Mascot GET Buscar por statu DEL Eliminar Mascota

MIA-Semana13-API > Eliminar Mascota

DELETE https://petstore3.swagger.io/api/v3/pet/11321

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 9 Test Results 200 OK · 18 ms · 382 B Save Response

JSON Preview Visualize

1 Pet deleted

Open Folder Terminal Console The coast is clear Cloud Agent Globals Vault Tools

GET posterior a la eliminación

The screenshot shows the Postman interface with the same workspace. The main panel shows a 'GET' request to 'https://petstore3.swagger.io/api/v3/pet/11321'. The response is a '404 Not Found' status with a body of 'Pet not found'.

Overview GET Consultar mascota p POST Crear mascota PUT Modificar Mascot GET Buscar por statu DEL Eliminar Mascota

MIA-Semana13-API > Consultar mascota por ID

GET https://petstore3.swagger.io/api/v3/pet/11321

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 9 Test Results 404 Not Found · 18 ms · 390 B Save Response

Thinking...

XML Preview Debug with AI

1 Pet not found

Open Folder Terminal Console Thinking... Cloud Agent Globals Vault Tools



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.

The screenshot displays a web browser window with the URL `petstore3.swagger.io`. The main content area shows the Swagger Petstore - OpenAPI 3.0 documentation. Below the documentation, there is a list of API endpoints with their respective methods and descriptions:

- PUT** `/pet`: Update an existing pet.
- POST** `/pet`: Add a new pet to the store.
- GET** `/pet/findByStatus`: Finds Pets by status.
- GET** `/pet/findByTags`: Finds Pets by tags.
- GET** `/pet/{petId}`: Find pet by ID.
- POST** `/pet/{petId}`: Updates a pet in the store with form data.
- DELETE** `/pet/{petId}`: Deletes a pet.
- POST** `/pet/{petId}/uploadImage`: Uploads an image.

On the right side of the browser, the Chrome DevTools Network tab is open, showing a list of requests. The selected request is `openapijson`, which is a GET request to `https://petstore3.swagger.io/api/v3/openapi.json`. The response status is `200 OK`. The response headers are visible, including `Content-Type: application/json` and `Date: Wed, 30 Sep 2026 13:36:51 GMT`.



petstore3.swagger.io/#/pet/getPetById

Todos los marcadores

NameDescription

petId * requiredID of pet to return
integer(\$int64)
(path)

petId

Responses

CodeDescriptionLinks

200successful operationNo links

Media typeapplication/json▼
Controls Accept header.

Example Value | Schema

```
{
  "id": 10,
  "name": "doggie",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

No links

400Invalid ID suppliedNo links

<>↺🔍petstore3.swagger.io/#/pet/addPet☆⌵Pregunta a GeminiA⋮—□×

U NotebookLMTodos los marcadores

Curl

```
cURL -X POST \
'https://petstore3.swagger.io/api/v3/pet' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "id": 11321,
  "name": "yerbi",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}'
```

Request URLhttps://petstore3.swagger.io/api/v3/pet

Server response

CodeDetails

500Error: response status is 500

Response body

```
{
  "code": 500,
  "message": "There was an error processing your request. It has been logged (ID: 348397bb6eaf5a34)"
}
```

Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 110
content-type: application/json
date: Wed, 30 Sep 2026 13:48:05 GMT
server: Jetty(9.4.53.v20231009)
```



amogollon-1-s-team.postman.co/workspace/MIA_2026~78e596e6-b59b-4925-977f-b4d5e16e2f76/http-request/...

Postman interface showing a GET request to `https://postman-echo.com/get?foo1=bar1&foo2=bar2`. The request is configured with query parameters:

Key	Value	Description
foo1	bar1	
foo2	bar2	

The response is a 200 OK status, 503 ms, 1.22 KB. The response body is JSON:

```
1 {
2   "args": {
3     "foo1": "bar1",
4     "foo2": "bar2"
5   },
6   "headers": {
7     "host": "postman-echo.com",
8     "x-forwarded-proto": "https",
9     "postman-token": "469fe3df-ice7-4e13-9f0a-c9284b9bbf81",
10    "user-agent": "PostmanRuntime/2.10.0",
11    "accept-encoding": "gzip, br",
12    "accept": "*/*",
13    "cache-control": "no-cache"
14  }
15 }
```